

UNIVERSIDAD DE SAN BUENAVENTURA CALI

Facultad de Ingeniería

INTRODUCCIÓN A INGENIERÍA DE SOFTWARE

Beatriz Eugenia Grass

PARCIAL 2

Ingeniería de requisitos

Health Corp

INTEGRANTES

Nicolas Lobo

Sara Sierra

Valeria Becerra

Juan José Bolívar

Miley Carrera

Hanna Valencia

1. Matriz de Riesgos

Diagnóstico general

El proyecto enfrenta un perfil de riesgo predominantemente alto, con 10 de 14 riesgos clasificados en ese nivel. Los riesgos más críticos no son técnicos sino humanos y de información: el equipo es inexperto en el sector salud, tiene dificultades de coordinación interna, y depende de fuentes de información limitadas. Sin embargo, cuenta con fortalezas sólidas: buena comunicación, motivación genuina y conocimiento base en gestión de proyectos, lo cual representa un punto de partida favorable.

Líneas de acción

1. Consolidar la estructura interna del equipo

La mayor concentración de riesgos está en la dinámica del grupo: inasistencias, conflictos, carga académica y falta de experiencia. Antes de arrancar el primer sprint, el equipo debe acordar un horario fijo de reuniones, definir roles claros según las fortalezas de cada integrante, firmar un acuerdo de compromiso y establecer un código de conducta básico. El GP es el principal responsable de sostener esta estructura durante todo el proyecto. Esta línea de acción atiende directamente cinco de los riesgos identificados y sienta las bases para que las demás acciones funcionen.

2. Cerrar la brecha de conocimiento del sector salud

El desconocimiento profundo del sector es el riesgo con mayor potencial de retraso. El equipo debe destinar los primeros dos sprints a una inmersión informada: revisar portales públicos del Ministerio de Salud y la Supersalud, construir un glosario del sector, y realizar al menos tres entrevistas con actores reales (médicos, administrativos, pacientes). El contacto familiar con la EPS debe activarse cuanto antes, ya que representa la fuente de información primaria más valiosa del proyecto. Esta acción también reduce la dependencia de una sola fuente, que es una amenaza identificada.

3. Proteger la documentación y los entregables

La pérdida de archivos es un riesgo alto y completamente evitable. Desde el primer día, todo el trabajo debe vivir en la nube (Google Drive o similar) con una política clara de nomenclatura y versionado. Nadie trabaja sobre el archivo original sin copia. Esta acción no requiere tiempo ni recursos significativos, pero su impacto en la continuidad del proyecto es alto.

4. Aprovechar las fortalezas como ventaja competitiva

La comunicación efectiva y la motivación del equipo no deben darse por sentadas: deben gestionarse activamente. Esto significa mantener reuniones con agenda, reconocer logros en cada encuentro y vincular el trabajo del proyecto con aprendizajes reales del sector. El conocimiento previo en gestión de proyectos debe traducirse en plantillas, estructuras y buenas prácticas documentadas desde el sprint 1, para que no quede en la cabeza de unos pocos.

5. Monitorear el entorno sin paralizarse

La reforma a la salud y la digitalización del sector son variables externas que el equipo no controla, pero sí puede anticipar. El GP debe hacer un seguimiento semanal breve del estado normativo y ajustar los supuestos del proyecto si hay cambios relevantes. Al mismo tiempo, la tendencia de telemedicina debe usarse como referente de diseño, no como dependencia.

2.Presupuesto:

El costo por hora de cada rol se calculó sumando todos los componentes salariales que le corresponden al empleador en Colombia:

Salario base 1.750.905, más subsidio de transporte de 249.906

Costos hora resultantes por rol:

- Desarrollador: \$441.864 COP/hora
- Product Owner: \$445.740 COP/hora

- Product Manager: \$733.533 COP/hora

Para el método de puntos de historia, el valor de cada punto se calculó así:

Valor del punto de historia = Costo total del equipo por mes / Puntos entregables por mes = 47.209.680 / 55 puntos por mes = 858.358 COP por punto

La disponibilidad varía según el rol:

- Desarrollador: 50% del día (8 horas efectivas de trabajo)
- Product Owner: 100% del día (16 horas)
- Product Manager: 100% del día (16 horas)

2.1 Presupuesto del proyecto:

Nómina del equipo (4 meses):

- 2 Product Owners: \$14.263.680/mes → \$57.054.720 total
- 1 Product Manager: \$11.736.528/mes → \$46.946.112 total
- 3 Desarrolladores: \$21.209.472/mes → \$84.837.888 total
- Subtotal recurso humano: \$188.838.720

Otros costos directos (durante los 4 meses):

- Servidor en la nube: \$168.000
- Servicio de videollamada (Daily.co o Agora): \$860.000
- Google Maps API: \$340.000
- Twilio SMS/Push notifications: \$340.000
- Subtotal otros costos: \$1.708.000

Reservas:

- Reserva de contingencia (horas extra en festivos): \$1.621.137
- Reserva gerencial (5% sobre el total): \$9.608.393

Utilidad del proyecto (10%): \$19.216.786

TOTAL DEL PRESUPUESTO DEL PROYECTO: \$201.776.250 COP

Costo total por módulo (estimado por puntos de historia):

- Telemedicina y Emergencias: \$35.192.678 — el más costoso por sus altos puntos de historia
- Sistema e Infraestructura: \$28.325.814 — incluye encriptación y notificaciones automáticas
- Módulo Médicos: \$23.175.666
- Autenticación y Perfil: \$17.167.160
- Módulo Administrativo: \$13.733.728
- Gestión de Citas: \$11.158.654
- Historial Clínico y Fórmulas: \$7.725.222

3.HUS

Método de estimación de esfuerzo Planning Poker

Para estimar el esfuerzo relativo de cada Historia de Usuario (HU) se utilizó la técnica de Planning Poker. Los 3 desarrolladores votaron de forma independiente usando la escala Fibonacci adaptada:

- 0, 1, 2, 3, 5, 8, 13 o más puntos

Se promediaron los votos de los tres desarrolladores y el resultado se mapeó a una escala de esfuerzo del 1 al 5:

- 0 puntos → Esfuerzo 0
- 1 a 2 puntos → Esfuerzo 1
- 3 puntos → Esfuerzo 2
- 5 puntos → Esfuerzo 3 o 4
- 8 puntos → Esfuerzo 4 o 5
- 13 o más puntos → Esfuerzo 5

Este esfuerzo normalizado (1–5) se utiliza luego en la fórmula de priorización.

La priorización se realizó usando la siguiente fórmula:

$$\text{Prioridad} = (\text{Urgencia} \times \text{Impacto}) / \text{Esfuerzo}$$

Donde:

- Urgencia (escala 1–5): qué tan pronto se necesita la funcionalidad para que el sistema sea funcional
- Impacto (escala 1–10): valor que la funcionalidad aporta al usuario o al negocio
- Esfuerzo (escala 1–5): derivado del Planning Poker; representa la complejidad técnica de la HU

Esta fórmula premia las HU que tienen alta urgencia e impacto pero bajo esfuerzo, es decir, las que generan más valor con menos inversión de tiempo. Las HU con esfuerzo alto bajan en el ranking aunque tengan impacto alto, pues representan mayor riesgo y costo temprano.

Ranking completo de las 23 HU por puntaje de prioridad:

1. HU13 — Como médico quiero ver mi agenda para organizarme en la jornada → Prioridad: 10.35
2. HU5 — Como paciente quiero saber la disponibilidad de médicos → Prioridad: 10.21
3. HU1 — Como paciente quiero registrarme en la app → Prioridad: 10.00
4. HU2 — Como paciente quiero iniciar sesión en mi cuenta → Prioridad: 10.00
5. HU7 — Como paciente quiero ver mi historial clínico → Prioridad: 10.00
6. HU19 — Como sistema quiero mandar fórmulas médicas → Prioridad: 9.88
7. HU21 — Como administrativo quiero aprobar medicamentos → Prioridad: 9.88
8. HU16 — Como médico quiero hacer incapacidades médicas → Prioridad: 9.80
9. HU22 — Como administrativo quiero consultar estado de trámites → Prioridad: 9.30
10. HU20 — Como sistema quiero monitorear acceso a historiales → Prioridad: 9.20
11. HU4 — Como paciente quiero agendar una cita médica → Prioridad: 9.12
12. HU8 — Como paciente quiero notificaciones de mis citas → Prioridad: 9.02
13. HU14 — Como médico quiero el historial clínico del paciente → Prioridad: 8.93
14. HU15 — Como médico quiero registrar diagnósticos → Prioridad: 8.50

15. HU12 — Como paciente quiero que mi info médica sea visible en emergencias → Prioridad: 8.29
16. HU11 — Como paciente quiero activar un botón de emergencia → Prioridad: 7.80
17. HU23 — Como administrativo quiero registrar nuevas clínicas → Prioridad: 7.70
18. HU6 — Como paciente quiero cancelar o reprogramar una cita → Prioridad: 7.38
19. HU18 — Como sistema quiero almacenar datos de forma segura → Prioridad: 7.20
20. HU17 — Como sistema quiero enviar notificaciones automáticas → Prioridad: 6.75
21. HU10 — Como paciente quiero telemedicina → Prioridad: 6.20
22. HU9 — Como paciente quiero conocer mis fórmulas médicas digitales → Prioridad: 6.09
23. HU3 — Como paciente quiero actualizar mis datos personales → Prioridad: 5.60

4.Mockups

Para la visualización de la interfaz se utilizó la Inteligencia Artificial V0 indicando las HU y sus criterios de aceptación.

[EPS Sura mockups - v0 by Vercel](#)

5.Patrón de Diseño

Se va a trabajar con el patrón de diseño **Observer** porque, a diferencia de otros patrones que resuelven problemas de construcción o de estructura de clases, Observer está diseñado específicamente para situaciones donde un cambio en un objeto necesita reflejarse de manera automática en otros objetos, sin que el objeto que cambia tenga que conocer quiénes son ni cuántos son los que deben reaccionar. Esto lo convierte en la solución ideal para problemas de:

- **Acoplamiento fuerte**
- **Consistencia entre objetos relacionados**
- **Escalabilidad de reacciones**

Esta elección no fue arbitraria. Al analizar las matrices de **Urgencia, Priorización e Impacto-Esfuerzo**, se identificó un patrón claro en la naturaleza del sistema: las funcionalidades más críticas del proyecto no son acciones aisladas de un solo módulo, sino eventos que desencadenan respuestas en cadena hacia múltiples actores del sistema al mismo tiempo. Un evento ocurre en un punto y múltiples partes deben reaccionar automáticamente, lo que describe con exactitud el problema que **Observer** fue creado para resolver.

Haciendo uso de este patrón, las soluciones reutilizables que se obtienen para el proyecto son:

- **Desacoplamiento entre módulos:** El módulo de citas no necesita saber que existen un módulo de notificaciones, uno de historial clínico y uno de agenda médica. Solo registra el cambio de estado y Observer avisa a todos. Si en el futuro se agrega un módulo de farmacia o laboratorio, el módulo de citas no se modifica.
- **Sincronización automática entre actores:** Cuando ocurre un evento, paciente, médico y administrativo se actualizan en el mismo instante sin que ninguno dependa del otro. No existe riesgo de que un actor quede desactualizado.
- **Gestión de estados de la cita:** Cuando una cita pasa de pendiente a confirmada, o de confirmada a cancelada, Observer dispara automáticamente todas las reacciones que ese cambio implica en cada módulo, sin lógica adicional en el módulo principal.
- **Escalabilidad sin modificar código existente:** Si aparece un nuevo actor o módulo que necesita reaccionar ante los mismos eventos, simplemente se registra como un nuevo observador. El código que ya funciona no se toca, reduciendo el riesgo de introducir errores.
- **Trazabilidad y auditoría:** Cada observador puede registrar independientemente los eventos que recibe, permitiendo construir un historial de cambios sin que esa responsabilidad recaiga sobre el módulo central. El módulo de historial clínico es simplemente un observador más que escucha y registra.
- **Integración con la pasarela de pago:** Cuando un pago se confirma, el cambio de estado notifica automáticamente al paciente, al médico y al sistema de facturación como observadores independientes del mismo evento

6. Técnica de Retrospectiva

La Estrella de Mar:

Se va a trabajar con la técnica de retrospectiva Estrella de Mar porque a diferencia de otras técnicas que solo preguntan qué salió bien y qué salió mal, esta técnica nos da 5 dimensiones diferentes para analizar cómo trabajamos como equipo, permitiéndonos ser mucho más específicos y no quedarnos solo en lo obvio.

Además, nuestro proyecto HealthCorp tiene muchas funcionalidades y roles diferentes, entonces necesitábamos una técnica que nos permitiera revisar el trabajo desde varios ángulos, no solo especificar lo que está bueno o malo.

Haciendo uso de esta técnica se concluyó lo siguiente:

Seguir haciendo

- Las reuniones fijas nos ayudaron a que todos estuvieran enterados de todo.
- Repartir el trabajo según lo que cada uno sabe hacer mejor.
- Definir bien los criterios de aceptación desde el inicio

Hacer mas

- Avisarle al equipo cuando algo no se entiende.
- Prototipar más seguido.
- Revisar entre todos antes de entregar.

Hacer menos

- Dejar tareas para el último día antes de la entrega.
- Reuniones donde terminamos hablando de otra cosa.
- Ponerse a discutir detalles pequeños cuando hay cosas más importantes pendientes.

Dejar de hacer

- Trabajar cada uno por su lado sin coordinarse con el resto.
- Dejar de cambiar cosas del documento sin avisarle a los demás.
- Dejar de entregar cosas sin haberlas leído primero como equipo.

Empezar a hacer

- Hacer que alguien simule ser paciente o médico antes del siguiente sprint.
- Que cada uno lea lo que hicieron los demás para entender el proyecto completo.
- Revisar lo que entregamos en el sprint anterior antes de arrancar el nuevo.